

User Stories

User Story #1 — Personal Username	
As a user, I want to have my personal username for recognizing me in the leaderboard.	
Validation: • Functional: Build a database to store usernames and passwords • Functional: Add a register page • Functional: Have a unique identifier for each user • Non-functional: Passwords must be encrypted • Non-functional: Username uniqueness must be enforced	Value: 155 Priority: Medium Estimation: 9h

User Story #2 — Secure Login	
As a user, I want to log in with a password so that my information is protected.	
Validation: • Functional: Add login page • Functional: Add token-based authentication • Non-functional: Login response under 2 seconds • Non-functional: Session tokens must expire securely	Value: 160 Priority: High Estimation: 10h

User Story #3 — Saved Progress	
As a user, I want my progress to be saved so I can disconnect and play later.	
Validation: • Functional: Add run-state persistence on server • Non-functional: Save operation under 2 seconds • Non-functional: Data must persist after crashes	Value: 165 Priority: High Estimation: 11h

User Story #4 — Session Resume	
As a user, if my session disconnects, I want to resume my run so I continue playing.	
Validation: • Functional: Add session system linked to server • Non-functional: Reconnection under 5 seconds • Non-functional: Mitigate progress loss	Value: 170 Priority: Medium Estimation: 12h

User Story #5 — Real-time game sync with server	
As a user, I want real-time communication so that the game feels like it is only running on my machine.	
Validation: • Functional: Add a WebSocket server • Functional: Add client-side listeners • Functional: Add event-sending mechanisms • Non-functional: Latency below 150ms	Value: 175 Priority: High Estimation: 13h

User Story #6 — Leaderboard	
As a user, I want to see the score of the best plays so that I can compare myself with others.	
Validation: • Functional: Add a leaderboard page • Functional: Add best-runs table • Functional: Add filters • Non-functional: Mitigate queries to the leaderboard db.	Value: 180 Priority: High Estimation: 14h

User Story #7 — Stats Comparison	
As a user, I want to compare my previous runs so I see my progress.	

Validation: • Functional: Add a user stats page • Non-functional: The page contains a runs started and time played stat • Non-functional: Page loads in under 2 seconds	Value: 185 Priority: High Estimation: 15h

User Story #8 — Best Run Record	
As a user, I want to see my record run to see how good I am.	
Validation: • Functional: Include best run stats • Functional: Add history of best 3 runs • Functional: Add exploit bar chart • Non-functional: Charts render smoothly	Value: 190 Priority: Medium Estimation: 16h

User Story #9 — Loan Tracker UI	
As a user, I want a page to check how much money I need to pay to the mafia so I know if my run is in danger.	
Validation: • Functional: Add bank page • Functional: Display current debt and balance • Functional: Add a server mechanism to track debt and rounds missing for paying • Non-functional: Information updates instantly	Value: 195 Priority: Medium Estimation: 17h

User Story #10 — Rounds Remaining UI Display	
As a user, I want to know how many rounds are left before my loan is overdue so I try to prolong them.	

Validation: • Functional: Display rounds remaining • Non-functional: Must always sync with game state	Value: 200 Priority: High Estimation: 8h

User Story #11 — Poker Tutorial	
As a user, I want a guide to poker so I learn the basic rules.	
Validation: • Functional: Add guided poker round • Non-functional: Tutorial under 10 minutes • Non-functional: Beginner-friendly	Value: 205 Priority: High Estimation: 9h

User Story #12 — Exploit Purchase Tutorial	
As a user, I want an intuitive way to buy exploits so I don't get confused and frustrated.	
Validation: • Functional: Explain exploits • Functional: Explain access methods • Non-functional: Clear and accessible UI	Value: 210 Priority: High Estimation: 10h

User Story #13 — Disabled Exploits	
As a user, I want unavailable exploits to be inaccessible so I know I can't use them.	
Validation: • Functional: Add disabled exploit indicators • Non-functional: Visual distinction must be obvious	Value: 215 Priority: High Estimation: 11h

User Story #14 — UI Feedback

As a user, I want a UI that correctly shows what I am doing so that I understand the current game state.

Validation:

- Functional: Clearly labeled buttons
- Non-functional: UI response under 100ms
- Non-functional: Consistent visual style

Value: 220

Priority: High

Estimation: 12h

User Story #15 — Exploit Descriptions

As a user, I want to see exploit descriptions so I understand all the exploits that I can use.

Validation:

- Functional: Add exploit descriptions to the card
- Non-functional: Make a clear description of what the exploit does when it is triggered.

Value: 225

Priority: High

Estimation: 13h

User Story #16 — Tools Visibility

As a user, I want to know where all my available tools are so that it feels intuitive to navigate the ui.

Validation:

- Functional: Add tutorial descriptions of tools
- Non-functional: Navigation must be intuitive

Value: 230

Priority: High

Estimation: 14h

User Story #17 — DMs Tutorial

As a user, I want to know what the DMs are for so I understand that I could check them to see the mafia's offers and warnings.

Validation:

- Functional: Include DMs tutorial
- Non-functional: Must be accessible anytime

Value: 235

Priority: Medium

Estimation: 15h

User Story #18 — Core Event Communication	
As a developer of the core, I want reliable event communication so that I can send information to the user.	
Validation: • Functional: Abstract WebSocket logic to a facade or adapter class • Non-functional: Fault-tolerant event delivery	Value: 240 Priority: High Estimation: 16h

User Story #19 — Modular Architecture	
As a developer, I want modular code so that it could be changed for different game modes.	
Validation: • Functional: Separate game modules • Functional: Reusable components • Functional: Clear interfaces • Non-functional: Documented modules • Non-functional: Scalable architecture	Value: 245 Priority: High Estimation: 17h

User Story #20 — Bet Management	
As a player, I want to place bets during each round so that I play poker.	
Validation: • Functional: Implement a betting system • Non-functional: rollback invalid bets or reject them	Value: 250 Priority: High Estimation: 8h

User Story #21 — Poker Actions	
As a player, I want to fold, call, raise, or check so that I can play my turn in poker.	
Validation: • Functional: Implement turn actions	Value: 255 Priority: High Estimation: 10h

• Non-functional: Action validation under 100ms • Non-functional: if a player raises the bet, all other players must have a new turn to see if they want to pay and continue playing or leave the round.	

User Story #22 — Difficulty Scaling	
As a player, I want increasingly difficult tables so that I feel that I am progressing in the game.	
Validation: • Functional: Increase AI complexity progressively • Non-functional: No unfair spikes • Non-functional: Balance AI so that the game can be won.	Value: 260 Priority: High Estimation: 10h

User Story #23 — Opponent Playstyles	
As a player, I want varied opponents so that rounds don't feel repetitive.	
Validation: • Functional: Aggressive/defensive/bluff AI • Non-functional: Decision under 8-9 seconds	Value: 265 Priority: Medium Estimation: 11h

User Story #24 — Frontend Event Reception	
As a front-end developer, I want to receive information constantly for synchronizing the client game with servers version.	
Validation: • Functional: WebSocket abstraction layer • Non-functional: Stable client-server sync	Value: 270 Priority: High Estimation: 12h

User Story #25 — Losing Chips	
As a player, I want to lose my chips when my account gets banned so that I can't abuse exploit use.	
Validation: • Functional: Implement chip reset on ban condition • Functional: Trigger run soft termination sequence • Non-functional: Penalty must apply instantly • Non-functional: Reset must preserve account integrity (money in the bank)	Value: 250 Priority: High Estimation: 4h

User Story #26 — Unlock new exploits	
As a player, I want to unlock permanent exploits, to feel long term progression.	
Validation: • Functional: Create exploit unlock system • Functional: Store upgrades across runs • Functional: Add upgrade selection interface • Non-functional: Unlock persistence must survive server restarts • Non-functional: Upgrade loading under 1 second	Value: 265 Priority: High Estimation: 3h

User Story #27 — Useful display/UI	
As a player, I want a clear display of chips, bets, and hand values, so that I can track the game state easily	
Validation: • Functional: Display chips, bets, and hand values clearly • Functional: Update interface in real time • Non-functional: UI updates under 100ms	Value: 200 Priority: Medium Estimation: 5h

• Non-functional: Readable across supported resolutions	

User Story #28 — Reusable code	
As a developer, I want reusable game logic components, so that maintenance is simpler.	
Validation: • Functional: Abstract shared poker logic • Functional: Create reusable utility classes • Non-functional: Components must be documented • Non-functional: Code reuse should reduce duplication	Value: 300 Priority: Medium Estimation: 20h

User Story #29 — Audio feedback	
As a player, I want sound effects for game actions so that gameplay feels immersive.	
Validation: • Functional: Add betting sound effects • Functional: Add win/loss audio cues • Functional: Add UI interaction sounds • Non-Functional: Audio playback latency under 100ms • Non-Functional: Volume must be adjustable	Value: 249 Priority: Medium Estimation: 8h

User Story #30 — Mafia	
As a core developer, I want to track the players' debt so that I can terminate the run if conditions are met.	

Validation: • Functional: Add a Mafia class that tracks remaining runs before payment and signals for a hard reset if necessary. • Functional: Add to the Mafia class a method to start listening to round end and <i>back the bet</i> if activated • Functional: Emit a signal that the player's chips or money was changed • Functional: Have a method to pay debt that reduces internal variables, changes rounds remaining. • Non-functional: on every payment the round remaining should change • Functional: if debt is paid, it should disable the trigger for terminating the run	Value: 195 Priority: High Estimation: 17h

User Story #31 — Casino	
As a core developer, I want to track the players' exploit use so that I can terminate the run if exploit use is abused.	
Validation: • Functional: Add a Casino class to the Player that tracks the rounds that have been passed and amount of exploits used • Non-functional: if the ratio of exploits used and rounds played is high, it should trigger a soft reset. • Functional: Add a method for exploits that change the deck; it checks if that action is valid and, if not, triggers a soft reset. Example: trying to change the	Value: 195 Priority: High Estimation: 17h

current player's hand to a card in another player's hand.	

User Story #32 — Communication with client	
As a developer of the session system, want all communication with the client or coming from the client to be JSON-based so that methods can be lost in the process	
Validation: - Functional: Make the events' payloads have the possibility to be sent in json format - Non-functional: reinterpret payloads if they depend on an instance of a class	Value: 300 Priority: Medium Estimation: 20h

User Story #33 — Client event emitting	
As a developer of the session system, want that the events sent by the client be validated so that I can process them without fear	
Validation: - Functional: Validate the events from the client are in the white list of what they could do - Non-functional: Users should not be able to directly add money to their instance or affect other players instances	Value: 300 Priority: High Estimation: 20h

User Story #34 — Player Statistics Monitoring	
As a developer, I want to analyze players' runs so that I can balance game correctly	
Validation: - Functional: Collect player performance data during each run - Functional: Store statistics such as win rate, average chips	Value: 295 Priority: High Estimation: 14h

earned, exploits used, and run duration • Functional: Display statistics through an analytics dashboard • Functional: Allow filtering data by difficulty level, player progression, and match outcomes • Functional: Generate reports for balance analysis • Non-Functional: Statistics must update in near real time • Non-Functional: Dashboard queries must load in under 3 seconds • Non-Functional: Data collection must not affect gameplay performance • Non-Functional: Analytics data must remain consistent and accurate across sessions	

User Story #35 — Visual feedback	
As a player, I want visual feedback to indicate something happened so that I can clearly understand game events.	
Validation: • Functional: Display visual indicators for player actions • Functional: Show notifications for wins, losses, and important events • Functional: Highlight changes in chips, bets, or game state • Functional: Trigger animations for key gameplay actions • Non-functional: Visual feedback must appear in under 100ms • Non-functional: Notifications must not obstruct important UI elements • Non-functional: Animations must remain smooth during gameplay	Value: 220 Priority: High Estimation: 5h

--

User Story #36 — Hover info	
As a player I want to hover over the exploits to see their art and description	
Validation: • Functional: Display exploit descriptions when hovered • Functional: Show exploit cost and effects • Functional: Indicate whether the exploit is available or unavailable • Functional: Update displayed information dynamically if exploit conditions change • Non-functional: Tooltip must appear in under 100ms • Non-functional: Information must be concise and easy to read • Non-functional: Tooltip must not block critical game interface elements	Value: 230 Priority: High Estimation: 7h

User Story #37 — Website	
As a user, I want a well-designed website that contains everything (statistics, the game, and all related features) so that I can access all game functions easily from one place.	
Validation: • Functional: Include access to the game client • Functional: Include player statistics page • Functional: Include leaderboard page	Value: 230 Priority: High Estimation: 7h

• Functional: Include tutorial section • Functional: Include account management features • Functional: Provide navigation between all sections • Non-functional: Website pages must load in under 3 seconds • Non-functional: Interface must be visually consistent across all sections • Non-functional: Navigation must be intuitive and easy to use • Non-functional: Website must be responsive across supported screen sizes	

User Story #38 — User deletion	
As an admin, I want to have the option to delete/add/ban a user if needed so that I can perform tests more efficiently	
Validation: • Functional: Implemented method to delete/add a user • Functional: Allow selecting and banning specific users • Functional: Allow reversing bans • Non-functional: The user must be notified of change in account status • Non-functional: Ban operations must not affect database integrity	Value: 250 Priority: Medium Estimation: 8h

User Story #39 — Poker Face
As a web developer, I want a component that handles the logic of buying exploits

Validation: • Functional: Component that triggers the client event of exploit buy to the server • Functional: The server hears the buy exploit event and executes it. • Functional: save all purchase exploits in an inventory class	Value: 250 Priority: Medium Estimation: 8h